



1 Contexte

Application nommée **LocDVD** de sélection de films et séries classés par catégorie.
Nous allons afficher des données enregistrées dans un fichier XML.

2 Remplir une ListView à partir d'un fichier XML

2.1 Créer la ListView

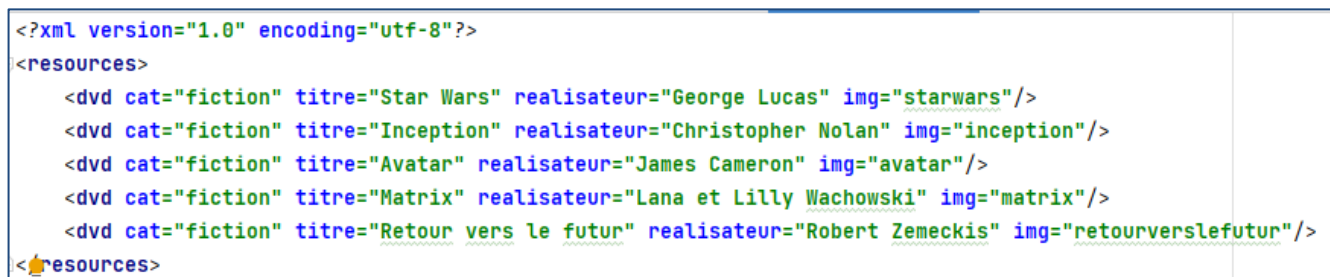
- Ouvrir le **layout activity_fiction**
- Ajouter une **ListView**
- Définir son id : **fiction_listview**

2.2 Création des données xml

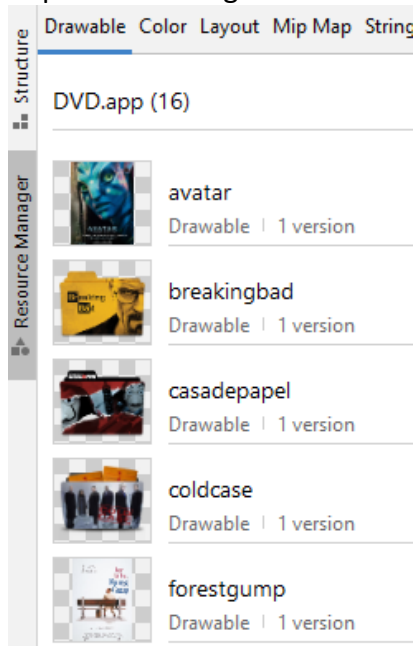
- Créer un sous répertoire **xml** dans le répertoire **res** (File / New / Android Resource Directory)



- Créer un fichier nommé : **liste_fictions.xml**



- Importer les images fournies des fictions dans le Ressources Manager : (Glisser/Déposer)



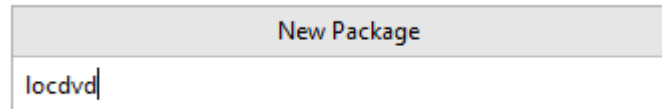


2.3 Lecture des données xml

Pour gérer ces données, nous aurons besoin d'objets de type **DvdModel**.

Nous allons donc créer un **package** contenant des classes complémentaires.

- Dans le répertoire **src/main/java**, créer un **nouveau package** nommé : **locdvd**



- Créer dans ce package une nouvelle **classe** nommée : **DvdModel**
- Créer **les attributs** descriptifs d'un dvd (idVideo, titreVideo, nomRealisateur, année, prixVideo, lienYoutube et imgVideo), un **constructeur par défaut** ainsi que tous les **accesseurs** et **mutateurs** :

```
package com.example.dvd.locdvd;

public class DvdModel {
    public int idVideo;
    public String titreVideo;
    public String nomRealisateur;
    public int annee;
    public double prixVideo;
    public int imgVideo;
    public String lienYoutube;

    public DvdModel() {
    }

    public int getIdVideo() { return idVideo; }

    public void setIdVideo(int idVideo) { this.idVideo = idVideo; }

    public String getTitreVideo() { return titreVideo; }

    public void setTitreVideo(String titreVideo) { this.titreVideo = titreVideo; }

    public String getNomRealisateur() { return nomRealisateur; }
```



- Ouvrir l'activité java nommée **Fiction**

Nous allons utiliser la classe **XmlPullParser**¹ pour lire les données du fichier **xml**.

- Écrire le traitement par **XmlPullParser** comme suit :

```
try {
    XmlPullParser xmlPullParser = getResources().getXml(R.xml.liste_fictions);
    //Tant que la fin de fichier non atteinte
    while (xmlPullParser.getEventType() != XmlPullParser.END_DOCUMENT) {
        //Si balise ouvrante
        if (xmlPullParser.getEventType() == XmlPullParser.START_TAG) {
            //Si la balise est DVD
            if (xmlPullParser.getName().equals("dvd")) {
                DvdModel uneFiction = new DvdModel();
                uneFiction.setTitreVideo(xmlPullParser.getAttributeValue(3));
                //Log.i("LocDVD", "Titre = " + uneFiction.getTitreVideo());
                uneFiction.setAnnee(Integer.parseInt(xmlPullParser.getAttributeValue(0)));
                //Log.i("LocDVD", "Année 3 = " + uneFiction.getAnnee());
                uneFiction.setNomRealisateur(xmlPullParser.getAttributeValue(2));
                //Log.i("LocDVD", "Realisateur = " + uneFiction.getNomRealisateur());

                //Récupération du nom de l'image dans drawable
                //String path = getPackageName() + ":drawable/" + xmlPullParser.getAttributeValue("imgVideo", "");
                int resID = getResources().getIdentifier(xmlPullParser.getAttributeValue(1), "drawable",
                    getPackageName());
                uneFiction.setImgVideo(resID);
                //Log.i("LocDVD", "Img 1= " + ":drawable/" + xmlPullParser.getAttributeValue(1));
            }
        }
        //Enregistrement suivant
        xmlPullParser.next();
    }
} catch (Exception e) {
    Log.i("LocDVD", "Erreurs trouvées = " + e.getMessage());
    e.printStackTrace();
}
```

Remarque : Vérifier l'affichage des informations dans la fenêtre Logcat

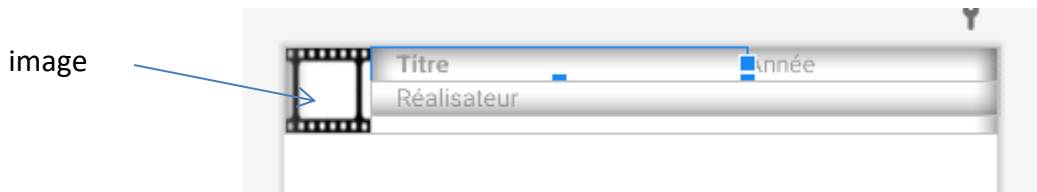
¹ API fournie par XmlPull.org et incorporée dans l'API Android.



2.3.1 Personnaliser une ListView

Nous allons présenter sur chaque ligne une image, le titre, l'année et le réalisateur. Pour cela, nous ne pouvons pas utiliser les modèles proposés par la ressource Android **android.R.layout**.

Nous allons donc créer notre propre modèle spécifique.

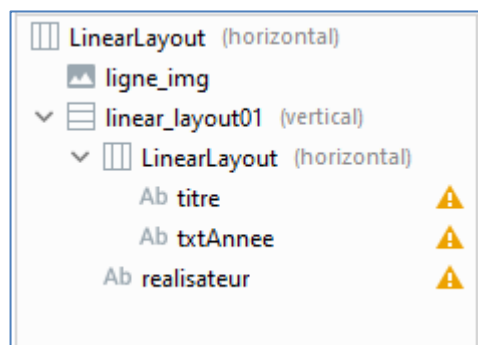


Attention : Le fichier **icon.png** doit être présent dans vos ressources dans **drawable**.

2.3.2 Création d'un layout spécifique

Pour cela, création d'un **LinearLayout** spécifique à chaque ligne de la **ListView**.

- Dans le dossier **res/layout**, créer un nouveau fichier **Xml / Layout XML File** nommé : **ligne.xml**.



```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/andr
    android:layout_width="fill_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal">

    <ImageView
        android:id="@+id/ligne_img"
        android:layout_width="50dp"
        android:layout_height="50dp"
        android:background="@drawable/icon"
        android:contentDescription="@string/iconeLigne"
        android:scaleType="centerInside" />
```



```
<LinearLayout
    android:id="@+id/linear_layout01"
    android:layout_width="fill_parent"
    android:layout_height="wrap_content"
    android:orientation="vertical">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:orientation="horizontal">

        <TextView
            android:id="@+id/titre"
            android:layout_width="match_parent"
            android:layout_height="fill_parent"
            android:layout_weight="1"
            android:hint="@string/titreLigne"
            android:paddingLeft="15dp"
            android:textColor="@color/black"
            android:textStyle="bold" />

        <TextView
            android:id="@+id/txtAnnee"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_weight="1.5"
            android:hint="Année" />
    </LinearLayout>

    <TextView
        android:id="@+id/realisateur"
        android:layout_width="fill_parent"
        android:layout_height="fill_parent"
        android:hint="@string/realisateurLigne"
        android:paddingLeft="15dp" />
</LinearLayout>
</LinearLayout>
```



2.3.3 Création d'un adaptateur spécifique

La **ListView** *fiction_listview* est spécifique et sera mise en œuvre en utilisant un adaptateur personnalisé pour l'objet **DvdModel**.

Cet adaptateur sera chargé de lire les attributs de chaque objet **DvdModel** de l'arrayList pour les transférer vers les objets graphiques correspondants dans le layout *ligne*

Création d'une nouvelle classe **DvdAdapter** dans le package **locdvd**

```
//Classe DvdAdapter qui dérive de la classe ArrayAdapter
//avec comme filtre l'objet DvdModel
public class DvdAdapter extends ArrayAdapter {

    //Constructeur
    public DvdAdapter(Context context, int textViewResourceId) {
        super(context, textViewResourceId);
    }

    @Override
    //Surcharge de la méthode getView pour prise en compte du LinearLayout spécifique ligne.xml
    public View getView(int position, View convertView, ViewGroup parent) {
        View result = convertView;
        if (convertView == null) {
            result = LayoutInflater.from(getContext()).inflate(R.layout.ligne, parent, false);
        }
        DvdModel dvd = (DvdModel) getItem(position);

        //Assigner aux textView les valeurs obtenues par les get de la classe Film
        TextView titre = (TextView) result.findViewById(R.id.titre);
        titre.setText(dvd.getTitreVideo());

        TextView annee = (TextView) result.findViewById(R.id.txtAnnee);
        annee.setText(String.valueOf(dvd.getAnnee()));

        TextView realisateur = (TextView) result.findViewById(R.id.realisateur);
        realisateur.setText(dvd.getNomRealisateur());

        ImageView imageview = (ImageView) result.findViewById(R.id.ligne_img);
        imageview.setImageResource(dvd.getImgVideo());

        return result;
    }

    //Notification à la vue principale des changements de la ListView
    public void updateData() { this.notifyDataSetChanged(); }
}
```

Dans le **layout activity_fiction.xml**

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".Series">
    <TextView
        android:id="@+id/listePolicier"
        android:layout_width="wrap_content"
        android:layout_height="41dp"
        android:layout_marginStart="16dp"
        android:layout_marginTop="72dp"
        android:layout_marginEnd="16dp"
        android:text="@string/txtListeSerie"
        android:textSize="30sp"
        app:layout_constraintBottom_toTopOf="@+id/LinearLayout2"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />
    <LinearLayout
        android:id="@+id/LinearLayout2"
        android:layout_width="409dp"
        android:layout_height="472dp"
        android:layout_marginStart="1dp"
        android:layout_marginTop="46dp"
        android:layout_marginEnd="1dp"
        android:layout_marginBottom="100dp"
        android:orientation="vertical"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintHorizontal_bias="0.0"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toBottomOf="@+id/listePolicier">
        <ListView
            android:id="@+id/serie_listview"
            android:layout_width="match_parent"
            android:layout_height="409dp">
        </ListView>
    </LinearLayout>
</androidx.constraintlayout.widget.ConstraintLayout>
```

Modifier la classe **Fiction**:

```
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_fiction);

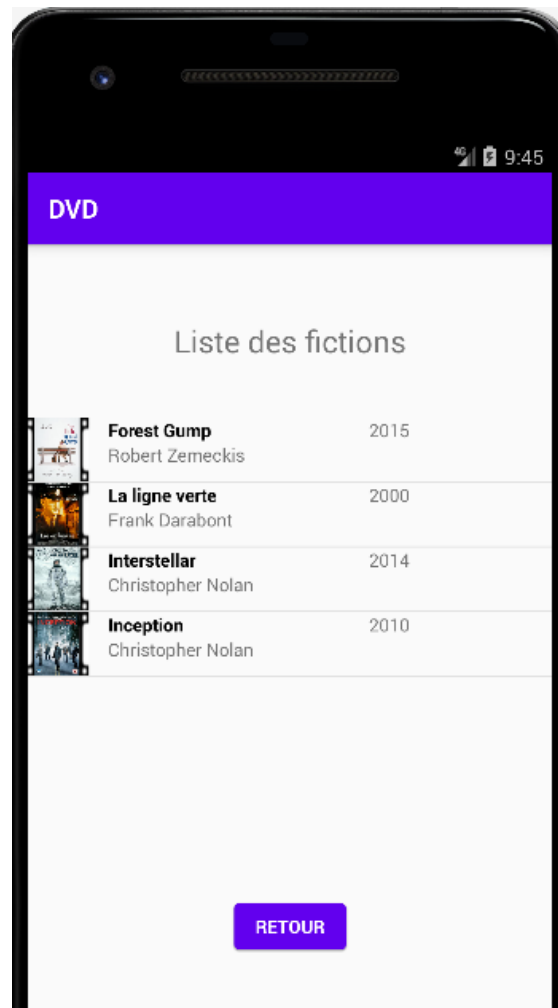
    //Variables privées pour la listView et le VideoAdapter
    ListView listeFictions = (ListView) findViewById(R.id.fictions_listview);
    DvdAdapter adapterDvd = new DvdAdapter(this, R.layout.ligne);

    try {
        XmlPullParser xmlPullParser = getResources().getXml(R.xml.liste_fictions);
        //Tant que la fin de fichier non atteinte
        while (xmlPullParser.getEventType() != XmlPullParser.END_DOCUMENT) {
            //Si balise ouvrante
            if (xmlPullParser.getEventType() == XmlPullParser.START_TAG) {
                //Si la balise est DVD
                if (xmlPullParser.getName().equals("dvd")) {
                    DvdModel uneFiction = new DvdModel();
                    uneFiction.setTitreVideo(xmlPullParser.getAttributeValue(3));
                    uneFiction.setAnnee(Integer.parseInt((xmlPullParser.getAttributeValue(0))));
                    uneFiction.setNomRealisateur(xmlPullParser.getAttributeValue(2));

                    //Récupération du nom de l'image dans drawable
                    String imgName = xmlPullParser.getAttributeValue(1);
                    int resID = getResources().getIdentifier(imgName, "drawable", getPackageName());
                    uneFiction.setImgVideo(resID);

                    adapterDvd.add(uneFiction);
                }
            }
            //Enregistrement suivant
            xmlPullParser.next();
        }
    } catch (Exception e) {
        Log.i("LocDVD", "Erreurs trouvées = " + e.getMessage());
        e.printStackTrace();
    }

    //Association de l'adapter à la listView
    listeFictions.setAdapter(adapterDvd);
}
```

Commentaires :

Cette étape a consisté à utiliser la classe **XmlPullParser** pour parser les données du fichier **XML**. Ces données seront stockées dans un premier temps dans une **ArrayList**. **XmlPullParser** est une API fournie par XmlPull.org et incorporée dans l'API Android.

Le constructeur de la classe **XmlPullParser** exploite la méthode **getResources** qui exploite elle-même la méthode **getXml**. La méthode **getXml** permet de récupérer les données des fichiers XML stockés dans le répertoire **res/xml**. (Il est donc impératif d'y stocker les fichiers XML de données).

Une fois les données du fichier **fictions.xml** récupérées dans un objet **XmlPullParser**, il suffit de parcourir l'objet pour récupérer les informations concernant chacune des séries de la catégorie Série. Chaque lot d'informations associé à une série est stocké dans un objet **DvdModel**. Ce code est implémenté dans la classe Fiction.

Tester l'activité. Visualiser le déroulement du programme grâce au **LogCat**. Commenter ensuite toutes les lignes **Log.i** dès que le programme fonctionne correctement.